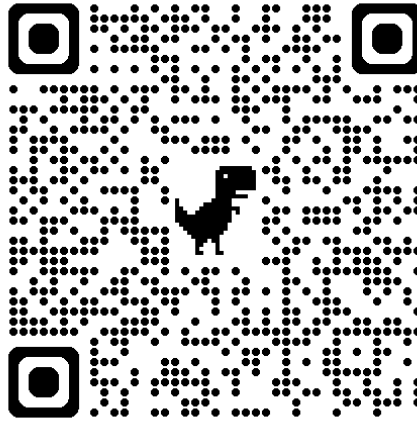


Lab Exercises for 2.086 Week 10

(Spring 2025)

<https://forms.gle/R1XPPcwyCJJU7YWh6>



In this lab, you will practice implementing machine learning methods to classify bicycles. There are two attendance questions, both requiring a screenshot upload.

Problem 1: Bicycle-Style Classification with k-NN

Classify bikes as Road (label = 1) vs. Mountain Bike (label = 2) using only the first two geometric features in the CSV.



1. Load data

```
rng(0)
fileName = 'Bikestyle_y.csv';

tbl      = readtable(fileName, 'PreserveVariableNames', true);
```

```

featIdx    = [1 6];                                % first-two columns as features
featNames = tbl.Properties.VariableNames(featIdx);

X          = tbl{:, featIdx};                       % numeric feature matrix (Nx2)
labels     = tbl{:, end};                           % last column = numeric labels 1/2

```

Labels 1 = Road bike, 2 = Mountain bike

```

%% Exploratory "pair-plot" (scatter-matrix) -----
pairIdx    = [1 2 3 4];                            % choose the columns you want
pairNames = tbl.Properties.VariableNames(pairIdx);
Xpair      = tbl{:, pairIdx};                       % numeric matrix (N x p)
group      = labels;                                % % 1 = Road (negative), 2 = MTB
(positive)

figure
% gplotmatrix syntax: X, Y, group, clr, sym, siz, doleg, dispopt
[h,ax,bigax] = gplotmatrix( Xpair, [], group, ...
                           'br', 'ox', [], [], 'hist');

title(bigax, 'Scatter matrix coloured by class');
legend(bigax, {'Road (1)', 'MTB (2)'}, 'Location', 'bestoutside');

```

Warning: Ignoring extra legend entries.

```

% Add axis-column labels manually
p = numel(pairIdx);
for r = 1:p
    for c = 1:p
        if r == p                                % bottom row → x-labels
            xlabel(ax(r,c), pairNames{c});
        end
        if c == 1                                % first col → y-labels
            ylabel(ax(r,c), pairNames{r});
        end
    end
end

title('Scatter matrix coloured by class');
legend({'Road (1)', 'MTB (2)'}, 'Location', 'bestoutside');

```

Warning: Ignoring extra legend entries.

```

title('Scatter matrix coloured by class');
legend({'Road (1)', 'MTB (2)'}, 'Location', 'bestoutside');

```

Warning: Ignoring extra legend entries.

Above scatterplot shows that some features are correlated with each other, such as seat tube length and saddle height.

Train / validation / test split (70 / 15 / 15)

```
cv1      = cvpartition(labels, 'HoldOut', 0.30);      % 70 % train
idxTrain = training(cv1);
idxTemp  = test(cv1);                                % remaining 30 %

cv2      = cvpartition(labels(idxTemp), 'HoldOut', 0.50);
idxVal   = false(size(labels)); idxVal(idxTemp) = test(cv2);      % 15 %
idxTest  = false(size(labels)); idxTest(idxTemp) = training(cv2); % 15 %

fprintf('Train: %d | Val: %d | Test: %d\n', ...
        nnz(idxTrain), nnz(idxVal), nnz(idxTest));
```

Train: 1731 | Val: 370 | Test: 371

Visualise training split

```
figure
gscatter(X(idxTrain,1), X(idxTrain,2), labels(idxTrain), 'br', 'ox');
xlabel(featNames{1}); ylabel(featNames{2});
legend({'Road (1)', 'MTB (2)'}, 'Location', 'best');
title('Training split – raw features');
```

k-NN without scaling (k = 5)

```
k      = 5;
mdl_raw = fitcknn(X(idxTrain,:), labels(idxTrain), 'NumNeighbors', k);
pred_raw = predict(mdl_raw, X(idxTest,:));

[Craw, order] = confusionmat(labels(idxTest), pred_raw);
confusionchart(Craw, order, ...
    'Title', '5-NN (no scaling)');
```

Interpret this confusion matrix.

```
TN = Craw(1,1); FP = Craw(1,2);
FN = Craw(2,1); TP = Craw(2,2);
acc_raw = (TP + TN) / sum(Craw(:));
tpr_raw = TP / (TP + FN);      % recall for Road class
fpr_raw = FP / (FP + TN);
```

```
fprintf('\nNO scaling → Acc = %.2f | TPR = %.2f | FPR = %.2f\n', ...
        acc_raw, tpr_raw, fpr_raw);
```

NO scaling → Acc = 0.86 | TPR = 0.65 | FPR = 0.07

k-NN with scaling

```
minX = min(X(idxTrain,:));
maxX = max(X(idxTrain,:));
rangeX = maxX - minX;
rangeX(rangeX==0) = 1; % avoid divide-by-zero if feature
constant

X_mm = (X - minX) ./ rangeX; % scale ALL rows with train stats
mdl_z = fitcknn(X_mm(idxTrain,:), labels(idxTrain), 'NumNeighbors', k);
pred_z = predict(mdl_z, X_mm(idxTest,:));

%% 5. k-NN with **z-score scaling**
% mu = mean(X(idxTrain,:));
% sig = std(X(idxTrain,:));
% X_z = (X - mu) ./ sig; % scale ALL rows with train stats
% mdl_z = fitcknn(X_z(idxTrain,:), labels(idxTrain), 'NumNeighbors', k);
% pred_z = predict(mdl_z, X_z(idxTest,:));

figure;
[Cz, order] = confusionmat(labels(idxTest), pred_z);
confusionchart(Cz, order, ...
    'Title','5-NN (scaling)');
```

```
TN = Cz(1,1); FP = Cz(1,2);
FN = Cz(2,1); TP = Cz(2,2);
acc_z = (TP + TN) / sum(Cz(:));
tpr_z = TP / (TP + FN);
fpr_z = FP / (FP + TN);

fprintf('WITH scaling → Acc = %.2f | TPR = %.2f | FPR = %.2f\n', ...
        acc_z, tpr_z, fpr_z);
```

WITH scaling → Acc = 0.87 | TPR = 0.70 | FPR = 0.08

All features and scaled features

Next, we test how the model works when it is using all eight features

```
% 1. Build full-feature matrix (N × p, p = #predictor columns)
featIdxAll = 1:(width(tbl)-1); % all but the label col
featNamesAll = tbl.Properties.VariableNames(featIdxAll);
```

```
Xall = tbl{:, featIdxAll}; % numeric matrix
```

% 2. k-NN WITHOUT scaling

```
k = 5;
mdl_full_raw = fitcknn(Xall(idxTrain,:), labels(idxTrain), ...
    'NumNeighbors', k);
pred_full_raw = predict(mdl_full_raw, Xall(idxTest,:));

[CrawF, order] = confusionmat(labels(idxTest), pred_full_raw);
confusionchart(CrawF, order, ...
    'Title','5-NN (all features, no scaling)');
```

```
TN = CrawF(1,1); FP = CrawF(1,2);
FN = CrawF(2,1); TP = CrawF(2,2);
acc_rawF = (TP + TN) / sum(CrawF(:));
tpr_rawF = TP / (TP + FN);
fpr_rawF = FP / (FP + TN);

fprintf('\nALL-feature, NO scaling → Acc = %.2f | TPR = %.2f | FPR =
%.2f\n', ...
    acc_rawF, tpr_rawF, fpr_rawF);
```

ALL-feature, NO scaling → Acc = 0.90 | TPR = 0.72 | FPR = 0.04

% 3. k-NN WITH min-max scaling

```
minX = min(Xall(idxTrain,:)); % training-only stats
maxX = max(Xall(idxTrain,:));
rangeX = maxX - minX;
rangeX(rangeX==0) = 1; % protect against zero-range cols

Xall_mm = (Xall - minX) ./ rangeX; % scale *all* rows the same way

mdl_full_mm = fitcknn(Xall_mm(idxTrain,:), labels(idxTrain), ...
    'NumNeighbors', k);
pred_full_mm = predict(mdl_full_mm, Xall_mm(idxTest,:));

[CmmF, order] = confusionmat(labels(idxTest), pred_full_mm);
confusionchart(CmmF, order, ...
    'Title','5-NN (all features, min-max scaling)');
```

```
TN = CmmF(1,1); FP = CmmF(1,2);
FN = CmmF(2,1); TP = CmmF(2,2);
acc_mmF = (TP + TN) / sum(CmmF(:));
```

```

tpr_mmF = TP / (TP + FN);
fpr_mmF = FP / (FP + TN);

fprintf('ALL-feature, MIN-MAX → Acc = %.2f | TPR = %.2f | FPR = %.2f\n',
...
acc_mmF, tpr_mmF, fpr_mmF);

```

ALL-feature, MIN-MAX → Acc = 0.92 | TPR = 0.86 | FPR = 0.06

% How much did accuracy / TPR / FPR change when adding more features?
 % Which feature subset would you keep if inference cost matters?

Hyperparameter tuning

```

%% -----
%% Hyper-parameter tuning: compare k = 3 vs. 5 vs. 7 vs. 9 (min-max-
scaled, all features)
% IMPORTANT: use **validation set only** for choosing k
% -----

kGrid = [3 5 7 9];
metrics = table('Size', [numel(kGrid) 4], ...
               'VariableTypes', {'double', 'double', 'double', 'double'}, ...
               'VariableNames', {'k', 'ValAcc', 'ValTPR', 'ValFPR'});

for ii = 1:numel(kGrid)
    k = kGrid(ii);

    mdl = fitcknn(Xall_mm(idxTrain,:), labels(idxTrain), ...
                  'NumNeighbors', k);

    predVal = predict(mdl, Xall_mm(idxVal,:));

    [Cval,~] = confusionmat(labels(idxVal), predVal);
    TN = Cval(1,1); FP = Cval(1,2);
    FN = Cval(2,1); TP = Cval(2,2);
    acc = (TP + TN) / sum(Cval(:));
    tpr = TP / (TP + FN);
    fpr = FP / (FP + TN);

    metrics{ii,:} = [k acc tpr fpr];
end

disp(metrics)

```

k	ValAcc	ValTPR	ValFPR
3	0.91892	0.82609	0.05036
5	0.91351	0.84783	0.064748
7	0.91351	0.83696	0.061151
9	0.91081	0.82609	0.061151

```
% Pick the k with highest validation accuracy (tie → smallest k)
[~,bestIdx] = max(metrics.ValAcc);
bestK       = metrics.k(bestIdx);
fprintf('\nChosen k = %d based on **validation** accuracy (0.2f)\n', ...
        bestK, metrics.ValAcc(bestIdx));
```

Chosen k = 3 based on **validation** accuracy (0.92)

```
%% Final model: train on Train+Val, evaluate once on Test
idxTrainVal = idxTrain | idxVal;

mdl_final = fitcknn(Xall_mm(idxTrainVal,:), labels(idxTrainVal), ...
                    'NumNeighbors', bestK);
predTest  = predict(mdl_final, Xall_mm(idxTest,:));

[Cbest, order] = confusionmat(labels(idxTest), predTest);
confusionchart(Cbest, order, ...
               'Title',sprintf('Final 5-NN (k = %d, all features, min-max
scaling)',bestK));
```

```
TN = Cbest(1,1); FP = Cbest(1,2);
FN = Cbest(2,1); TP = Cbest(2,2);
acc_best = (TP + TN) / sum(Cbest(:));
tpr_best = TP / (TP + FN);
fpr_best = FP / (FP + TN);

fprintf('TEST performance (k = %d) → Acc = 0.2f | TPR = 0.2f | FPR =
0.2f\n', ...
        bestK, acc_best, tpr_best, fpr_best);
```

TEST performance (k = 3) → Acc = 0.93 | TPR = 0.86 | FPR = 0.04

```
%% -----
% Hyper-parameter search (here, choosing k) used only Train and Val.
% The Test set stayed untouched until a single final evaluation.
```

Feature Engineering

```
% -----
% 1) ***Feature creation*** - add domain-inspired ratios/differences
% -----
tbl.SeatLengthByDiameter = tbl("Seat tube length") ./ ...
                             tbl("Seat tube diameter");           % slimmess
```

```

tbl.WheelToFrameRatio = tbl("Wheel diameter front") ./ ...
                        tbl("Seat tube length");           % geometry
tbl.SaddleOffset      = tbl("Saddle height") - ...
                        tbl("Seat tube length");           % rider fit
% New variables are appended to the *end* of the table.

% -----
% 2) Feature selection - rank by |corr| with the label
% • Convert any NaN correlations to 0 (constant or non-numeric vars)
% -----
labelVar = "Label"; % change if needed
labelIdx = find(strcmp(tbl.Properties.VariableNames, labelVar));

candIdx = setdiff(1:width(tbl), labelIdx); % predictor columns
XallRaw = tbl{:, candIdx}; % numeric matrix
corrs = abs(corr(XallRaw, labels, 'Rows', 'complete'));
corrs(isnan(corrs)) = 0; % replace NaNs by 0

[~, ranked] = sort(corrs, 'descend');

topK = 8; % keep top-k
selIdx = candIdx(ranked(1:min(topK, numel(ranked)))));

fprintf('Selected top-%d features (by |corr|):\n', numel(selIdx));

```

Selected top-8 features (by |corr|):

```
disp(tbl.Properties.VariableNames(selIdx))
```

```

{'Teeth on chainring 0'    }
{'Dropout spacing'        }
{'Seat tube length'       }
{'SaddleOffset'           }
{'Wheel diameter front'   }
{'Head tube length textfield'}
{'Saddle height'          }
{'Stack'                  }

```

```

Xsel = tbl{:, selIdx}; % feature matrix

% -----
% 3) ***Normalisation*** - min-max to [0,1] (train stats only)
% -----
cv1 = cvpartition(labels, 'HoldOut', 0.30); % 70 % train
idxTrain = training(cv1);
idxTemp = test(cv1); % 30 % temp

cv2 = cvpartition(labels(idxTemp), 'HoldOut', 0.50);
idxVal = false(size(labels)); idxVal(idxTemp) = test(cv2); % 15 %
idxTest = false(size(labels)); idxTest(idxTemp) = training(cv2); % 15 %

minX = min(Xsel(idxTrain, :));

```

```

maxX = max(Xsel(idxTrain,:));
rangeX      = maxX - minX;
rangeX(rangeX==0) = 1;                                % guard 0 range
Xsel_mm = (Xsel - minX) ./ rangeX;                    % scaled matrix

%% -----
%% k-NN experiment with engineered + selected + scaled features
%% *k = 3* (hyper-parameter chosen on **validation** only)
%% -----
kGrid = [3 5 7];
bestAcc = -inf; bestK = 3;

for kk = kGrid
    mdlTmp = fitcknn(Xsel_mm(idxTrain,:), labels(idxTrain), ...
                     'NumNeighbors', kk);
    predVal = predict(mdlTmp, Xsel_mm(idxVal,:));
    accVal = mean(predVal == labels(idxVal));

    if accVal > bestAcc
        bestAcc = accVal; bestK = kk;
    end
end
fprintf('Chosen k = %d from validation (Acc = %.2f)\n', bestK, bestAcc);

```

Chosen k = 7 from validation (Acc = 0.93)

```

% Final model on Train+Val → evaluate once on Test
idxTrainVal = idxTrain | idxVal;
mdl_final = fitcknn(Xsel_mm(idxTrainVal,:), labels(idxTrainVal), ...
                   'NumNeighbors', bestK);
predTest = predict(mdl_final, Xsel_mm(idxTest,:));

[Cfe, order] = confusionmat(labels(idxTest), predTest);
confusionchart(Cfe, order, ...
               'Title', sprintf('Final k-NN (k = %d, engineered + scaled)', bestK));

```

```

TN = Cfe(1,1); FP = Cfe(1,2);
FN = Cfe(2,1); TP = Cfe(2,2);
acc_fe = (TP + TN) / sum(Cfe(:));
tpr_fe = TP / (TP + FN);
fpr_fe = FP / (FP + TN);

fprintf('TEST performance → Acc = %.2f | TPR = %.2f | FPR = %.2f\n', ...
        acc_fe, tpr_fe, fpr_fe);

```

TEST performance → Acc = 0.90 | TPR = 0.75 | FPR = 0.05

Problem 2: MATLAB CLASSIFICATION TOOLBOX

```
%%%%% Use Matlab toolbox to run several k nearest neighbor classification
%%%%% using all the features.
```

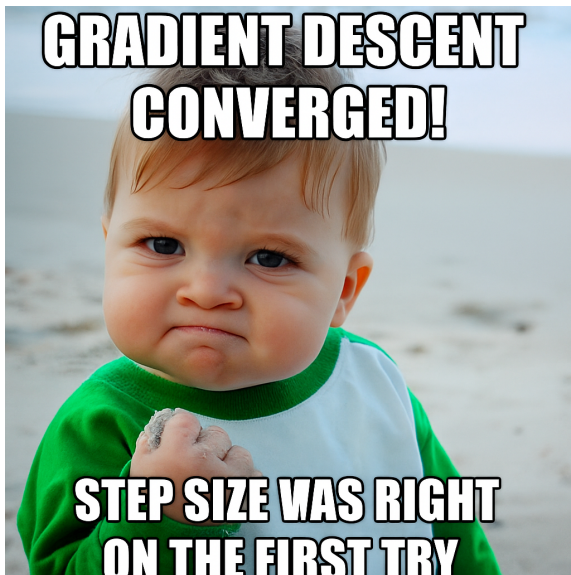
```
trainedKNNModel1.predictFcn
```

- **Apps Classification Learner** → **New Session** → **From File**
- Set **Response** column, **Holdout = 30 %**
- **Model Type**: train **All k-NN** presets
- **Compare** models → inspect confusion matrix, ROC, metrics
- **Export** best model → test on true 15 % set in code (optional)
- **Generate Function** for reproducible script

Attendance question 1: Upload the image of the highest accuracy you obtained using the toolbox

Attendance question 2: Upload an image of a meme related to 2.086 created by AI

Meme Challenge



Because a dash of creativity makes the stats stick—wrap up by memeing any concept you met in 2.086

```
% Let's see your creativity.
% Your goal is to use an AI tool (e.g., Chatgpt, Gemini) that you have
% access to and submit a meme
% related to any component of the course 2.086.

% Most students should have access to large language models.
```

% See Generative AI tools available at MIT <https://ist.mit.edu/ai-tools>. We believe that
% most MIT students should have free access to:
% - ChatGPT “4o” student free plan and the new o-series “mini” reasoning
% models (both accept images/PDFs) <https://chatgpt.com/students>
% - Microsoft Copilot <https://copilot.microsoft.com/>
% - Claude 3.7 Sonnet (limited quota) and Claude 3.5.
% - Gemini 2.0 via your Google account. See <https://gemini.google/students/?hl=en>.